

AXI-Based Multi-Master SoC

**with Round-Robin Arbitration, Asynchronous FIFO Bridging, and
Dual UVM / Class-Based Verification Environments**

A parameterized RTL interconnect that arbitrates three independent AXI-style masters onto a single memory-mapped AXI slave through a gray-code-synchronized asynchronous FIFO, verified with constrained-random UVM and class-based SystemVerilog testbenches achieving 100% functional coverage.

Author: Gautam Raj Chhabra

B.Tech, Electronics and Communication Engineering (ECE)

Expected Graduation: 2028

1. Title & Executive Summary

Project: AXI-Based Multi-Master SoC — Round-Robin Arbitration, Asynchronous FIFO Bridging, and Dual Verification Environments (UVM + Class-Based SystemVerilog).

This repository implements a small, self-contained System-on-Chip (SoC) fabric in which three independent AXI-style master state machines contend for a single AXI-style memory-mapped slave through a fair round-robin arbiter and a dual-clock-capable asynchronous FIFO. The design demonstrates core SoC interconnect concepts — handshake-based valid/ready protocols, arbitration fairness, clock-domain-safe pointer synchronization, and transaction packetization — while remaining compact enough to verify exhaustively. The project includes two independent, functionally equivalent verification environments (a UVM environment and a lightweight mailbox-based class environment) so that the design's correctness is cross-checked by two different methodologies, and both report 100% functional coverage with zero assertion failures across randomized and directed regressions.

The README that follows documents the project exactly as recommended for RTL design and verification portfolios: overview, features, repository layout, architecture, design specifications, verification methodology, test plan, results (with space reserved for simulation and waveform screenshots), build/run instructions, tools used, future work, and author details.

2. Project Overview

Modern SoCs multiplex several bus masters (CPUs, DMA engines, accelerators) onto a shared memory or peripheral space using an AXI-style interconnect. This project models that problem at a manageable scale: three `axi_master` instances independently alternate between write and read transactions, a `rr_arbiter` grants bus ownership fairly using a round-robin (last-grant-rotate) policy, and a single `axi_slave` services the granted transaction against a 256-entry, 32-bit-wide memory array. Because the masters and the slave operate conceptually as separate agents, transaction requests are packetized (address, data, read/write flag, and a valid bit) and pushed through an asynchronous FIFO before being unpacked and driven onto the slave interface — a pattern representative of how real SoCs decouple producer and consumer timing domains even when, as here, they share a common clock for simulation simplicity.

The core engineering challenge solved by this project is correctness under contention: multiple masters may request the bus in the same cycle, and the design must guarantee (a) exactly one grant at a time (mutual exclusion), (b) that a granted master retains the bus until its transaction fully completes (no starvation mid-transaction), (c) that the FIFO never loses or duplicates a packet across the write/read clock domains, and (d) that every AXI-style channel obeys the valid-must-remain-asserted-until-ready handshake rule. These properties are encoded directly into the RTL as SystemVerilog Assertions (SVA) and are additionally checked functionally by two independent testbenches.

The verification objective was to reach high confidence in protocol compliance and data integrity using constrained-random stimulus across the full 32-bit address space (partitioned into low/mid/high regions) and the full 32-bit data space (partitioned into four quadrants), backed by directed corner-case tests at address boundaries (0x00000000 and 0xFFFFFFFF) and back-pressure scenarios (BREADY/RREADY deasserted). Both the UVM and class-based environments implement generator/driver/monitor/scoreboard/coverage components and were run to 100% functional coverage with zero UVM_ERROR/UVM_FATAL counts.

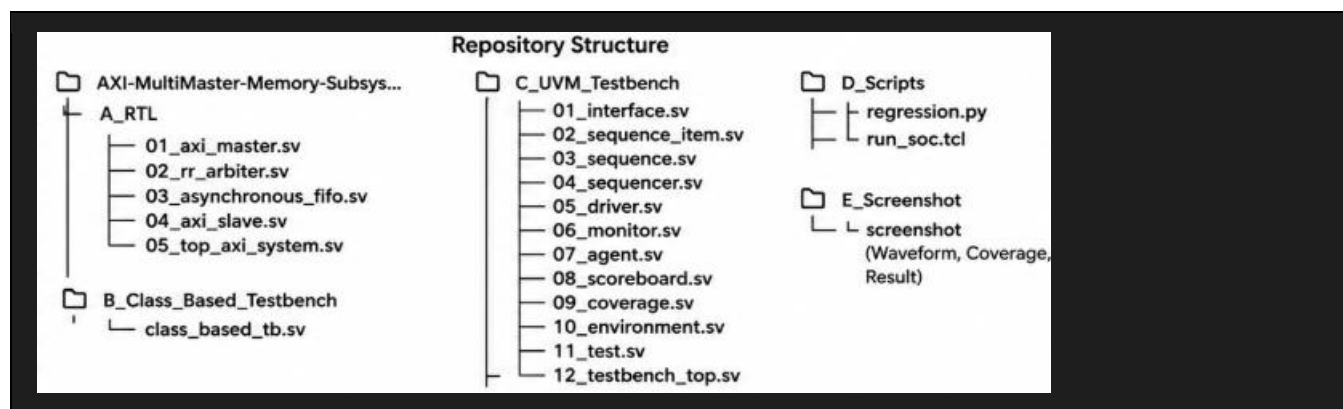
3. Key Features

- Three independent AXI-style master FSMs (`axi_master`) that autonomously alternate between write and read transactions on every arbitration cycle.
- Round-robin bus arbiter (`rr_arbiter`) with one-hot grant encoding, busy-hold-until-done semantics, and rotating priority across three requesters.

- Dual-clock asynchronous FIFO (asyn_fifo) using binary-to-gray pointer conversion and two-flop synchronizers for safe cross-domain full/empty detection.
- Single AXI-style memory-mapped slave (axi_slave) with a 256 x 32-bit memory array and independent write-response and read-data FSMs.
- 67-bit transaction packetization (valid | write-flag | reserved | 32-bit address | 32-bit data) so a single FIFO channel can carry both write and read requests.
- Top-level packet-tracking FSM (top_axi_system) that pops one FIFO packet at a time and drives the slave's AW/W/AR channels until each handshake completes, then clears state for the next packet.
- Embedded SystemVerilog Assertions for AWVALID/WVALID/BVALID/RVALID stability, one-hot grant, grant-hold-until-done, and FIFO full/empty pointer protection.
- Two parallel, functionally equivalent testbenches: a full UVM environment (agent, sequencer, driver, monitor, scoreboard, coverage) and a lightweight mailbox-based class environment.
- Constrained-random stimulus: address-region distribution (low/mid/high), data-quadrant distribution, and 80/20 ready-signal distributions to exercise back-pressure.
- Directed regression suite covering address-0, address-boundary, and multiple read/write corner cases.
- 100% functional coverage achieved on write/read type, address region, data quadrant, and ready-signal cross-coverage bins in both environments.
- Python + Tcl regression automation for headless Vivado batch simulation with pass/fail reporting.

4. Repository Structure

The project is organized to separate synthesizable RTL, the two verification environments, automation scripts, and documentation:



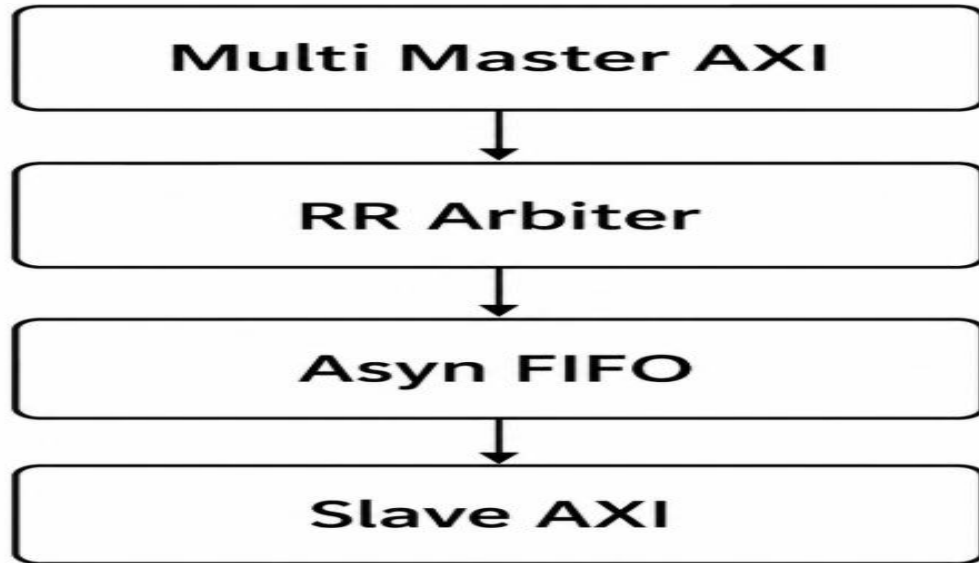
5. Architecture

5.1 Module Hierarchy

The `top_axi_system` module instantiates three `axi_master` instances (`master0`, `master1`, `master2`), a single `rr_arbiter`, one `asyn_fifo`, and one `axi_slave`. Each master independently drives its own AW/W/B and AR/R channel set; a request vector (`req[2:0]`) is derived by OR-ing each master's AWVALID/ARVALID signals. The arbiter observes this vector together with a `tx_done` signal (asserted when the currently granted master's B or R channel completes its handshake) and produces a one-hot grant vector.

Once a master is granted, its address/data channels are muxed onto a common set of wires (`awaddr_mux`, `wdata_mux`, `araddr_mux`, ...) and packed into a 67-bit word that is pushed into the asynchronous FIFO whenever `fifo_wr_en` is asserted (i.e., a valid request exists and the FIFO is not full). A packet-tracking sequential block on the read side pops one packet

at a time from the FIFO, holds it in `current_pkt` while driving the AXI slave's AW/W or AR channel(s), tracks partial handshake completion with `pkt_aw_done` / `pkt_w_done` / `pkt_ar_seen`, and clears state only once the full write (address+data+response) or read (address+data) sequence has completed — at which point the next packet may be popped.



5.2 Data Flow

- Master issues AWVALID/ARVALID → arbiter observes `req[2:0]` and asserts a one-hot grant.
- Granted master's address/data are muxed onto shared wires and packed into a 67-bit FIFO word.
- Packet is written into the asynchronous FIFO (`wr_clk` domain) once space is available (`!fifo_full`).
- Packet FSM pops the FIFO (`rd_clk` domain), reconstructs write/read flag, address, and data.
- Packet FSM drives the AXI slave's AW/W or AR channel(s) until the slave asserts BVALID or RVALID.
- Completion (`write_complete` / `read_complete`) routes BVALID/RVALID and RDATA back to the originally-granted master and clears `tx_done` so the arbiter can rotate to the next requester.

5.3 Clocking and Reset Strategy

All functional blocks in the top-level share a single system clock (`clk`) for simulation, though the `asyn_fifo` is designed with fully independent `wr_clk`/`rd_clk` ports and gray-code pointer synchronization so it would function correctly even if the write and read sides were later separated into distinct clock domains. The masters, arbiter, and slave use a synchronous active-high reset (`reset`), while the asynchronous FIFO uses an active-low reset convention (`wr_resetsn` / `rd_resetsn`), and the top level generates these from the same reset input via inversion.

6. Design Specifications

Parameter / Interface	Value / Description
ADDR_WIDTH	32 bits (all AXI-style address channels)
DATA_WIDTH	32 bits (all AXI-style data channels)
Number of Masters	3 (master0, master1, master2)

Parameter / Interface	Value / Description
Arbitration Policy	Round-robin, one-hot grant, rotates last_grant among 3 requesters
FIFO Depth	16 entries (addr_width = 4)
FIFO Word Width	67 bits: [66]=valid, [65]=write-flag, [64]=reserved, [63:32]=address, [31:0]=data
FIFO Pointer Scheme	Binary pointer + Gray-code conversion, 2-flop synchronizers across domains
Slave Memory Depth	256 words (indexed via addr_reg[9:2], i.e. word-aligned 1 KB space)
Reset Convention	Active-high (masters / arbiter / slave); active-low (async FIFO)
Protocol Style	Simplified AXI4-Lite-like: separate AW/W/B and AR/R channels with VALID/READY handshake
Write Response	Single BVALID/BREADY handshake per write (no BRESP encoding modeled)
Read Data	Single RVALID/RREADY handshake per read (no RRESP encoding modeled)

Assumptions and limitations: the design models the handshake and arbitration behavior of an AXI-style fabric rather than the full AXI4 specification — burst transfers,WSTRB byte-enables, protection/QoS/ID signaling, and out-of-order responses are intentionally out of scope so that the interconnect and arbitration logic can be verified exhaustively within a reasonable regression time. The write and read address spaces are treated identically (no separate memory maps per master); the slave decodes only the low-order address bits into its 256-word array, so addresses are effectively taken modulo the memory depth.

7. Verification Methodology

7.1 Assertions (SystemVerilog Assertions / SVA)

Protocol-level properties are embedded directly in the RTL and checked continuously during simulation:

Property	Module	Checks
p_awvalid_until_ready / p_wvalid_until_ready	axi_master	AWVALID/WVALID must not deassert before the corresponding READY is observed
p_bvalid_until_bready / p_rvalid_until_rready	axi_slave	BVALID/RVALID must remain asserted until BREADY/RREADY is sampled
p_onehot_grant	rr_arbiter	The grant vector must always be one-hot or zero (mutual exclusion)
p_hold_grant_until_done	rr_arbiter	grant must remain stable while busy and tx_done is not yet asserted
p_no_write_when_full	asyn_fifo	wr_ptr_bin must not change while full && wr_en
p_no_read_when_empty	asyn_fifo	rd_ptr_bin must not change while empty && rd_en

7.2 UVM Verification Environment

The UVM environment (tb/uvvm) follows the standard agent-based topology. `axi_transaction` extends `uvm_sequence_item` and carries randomizable `AWVALID/ARVALID/WVALID/BREADY/RREADY` fields plus region selectors (`aw_region`, `ar_region`) that steer `post_randomize()` address generation into low (`0x0000_0000+`), mid (`0x0000_1000+`), or high (`0x0001_0000+`) windows; `wdata` is constrained to a four-way quadrant distribution. `axi_sequence` issues 200 constrained-random transactions followed by 12 directed write/read transactions at fixed corner-case addresses with specific `BREADY/RREADY` values. `axi_driver` drives the virtual interface for each transaction and deasserts `VALID` after one clock; `axi_monitor` passively samples the bus and publishes observed transactions on two analysis ports — one to `axi_scoreboard` (which classifies and logs `WRITE/READ PASS` or `FAIL`) and one to `axi_coverage` (which samples a covergroup with per-signal and cross bins). `axi_env` wires the agent, scoreboard, and coverage together; `axi_test` raises/drops the UVM phase objection around sequence execution; `tb_top` instantiates the DUT, generates the clock/reset, and calls `run_test("axi_test")`.

7.3 Class-Based (Non-UVM) Verification Environment

A second, independent environment (tb/class_based) reimplements the same verification intent using plain SystemVerilog classes and mailboxes instead of the UVM base classes: generator produces 30,000 constrained-random `soc_tx` transactions plus the same 12 directed corner cases and pushes them through a `gen2drv` mailbox; driver pulls transactions and drives the `soc_if`; monitor samples the bus and forwards observed transactions to `mon2scb` and `mon2cov` mailboxes; scoreboard and coverage consume those mailboxes exactly as their UVM counterparts do. environment forks all components with `join_none`, and test wraps environment execution and coverage reporting. Running the same design intent through two independently coded environments increases confidence that any pass/fail result reflects genuine DUT behavior rather than a testbench-specific artifact.

7.4 Scoreboarding Strategy

Both scoreboards apply the same lightweight, self-checking rule per observed transaction: a transaction with `awvalid` asserted is classified as a `WRITE` and is reported as `WRITE PASS` if `wvalid` was also observed asserted (`WRITE FAIL` / "WDATA Missing" otherwise); a transaction with `arvalid` asserted is classified as a `READ` and reported as `READ PASS`. Every classified transaction also feeds the coverage model, so functional coverage and pass/fail checking are derived from the same monitored transaction stream.

8. Test Plan

Test Category	Description	Volume	Expected Result
Constrained-random R/W (UVM)	50/50 write-read split, address-region and data-quadrant distributions, 80/20 ready distributions	200 transactions	All WRITE/READ transactions classified PASS; assertions hold
Constrained-random R/W (class-based)	Same distributions as UVM sequence, larger volume for deeper regression	30,000 transactions	All WRITE/READ transactions classified PASS; assertions hold
Directed address-0 / low-region	Write then read back at <code>0x00000000</code> and <code>0x00000004</code> with <code>BREADY</code> toggled	2 write + 2 read	PASS, exercises reset-adjacent address
Directed mid-region	Write/read at <code>0x00008000</code> / <code>0x00008004</code>	2 write + 2 read	PASS, exercises mid address window

Test Category	Description	Volume	Expected Result
Directed boundary / high address	Write/read at 0xFFFFFFFFC and 0xFFFFFFFF8 (top of address space)	2 write + 2 read	PASS, exercises address wrap into 256-word memory
Back-pressure (BREADY/RREADY = 0)	Directed transactions issued with response-ready deasserted on alternating cases	Included above	BVALID/RVALID held stable per SVA until READY asserts
Arbitration fairness / contention	Multiple masters request simultaneously under random stimulus	Continuous across regression	Grant one-hot and rotates fairly; p_onehot_grant / p_hold_grant_until_done never fail
FIFO full / empty protection	Exercised naturally as depth-16 FIFO fills under back-to-back requests	Continuous across regression	p_no_write_when_full / p_no_read_when_empty never fail

9. Results

9.1 Functional Coverage Summary

Both the UVM and class-based environments reached full functional coverage:

Coverpoint / Cross	Coverage
cp_write (AWVALID asserted / not asserted)	100.00%
cp_read (ARVALID asserted / not asserted)	100.00%
cp_awaddr (low / mid / high write address)	100.00%
cp_araddr (low / mid / high read address)	100.00%
cp_wdata (four data quadrants)	100.00%
cp_bready (write response ready / not ready)	100.00%
cp_rready (read data ready / not ready)	100.00%
Overall FUNCTIONAL COVERAGE	100.00%

9.2 UVM Report Summary

```

--- UVM Report Summary ---
** Report counts by severity
UVM_INFO      : 2
UVM_WARNING   : 0
UVM_ERROR     : 0
UVM_FATAL     : 0

```

Zero UVM_ERROR and UVM_FATAL counts across the full regression confirm that no assertion or scoreboard mismatch occurred. Representative scoreboard output (both environments print an identical WRITE/READ TRANSACTION + RESULT format):

```

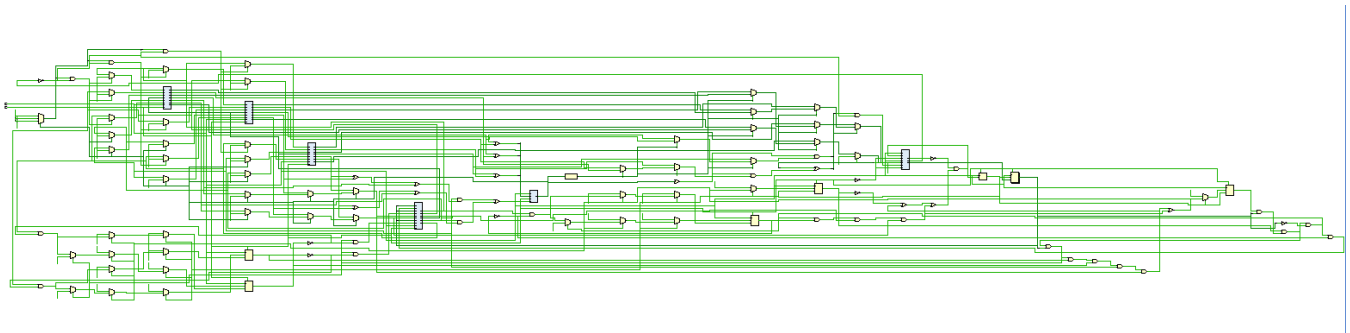
SCOREBOARD
WRITE TRANSACTION
Address : 000008c4
Data    : c7b8e049
RESULT  : WRITE PASS
SCOREBOARD
READ TRANSACTION
Address : 00000d24
RESULT  : READ PASS

```

9.3 Waveform and Tool Screenshots

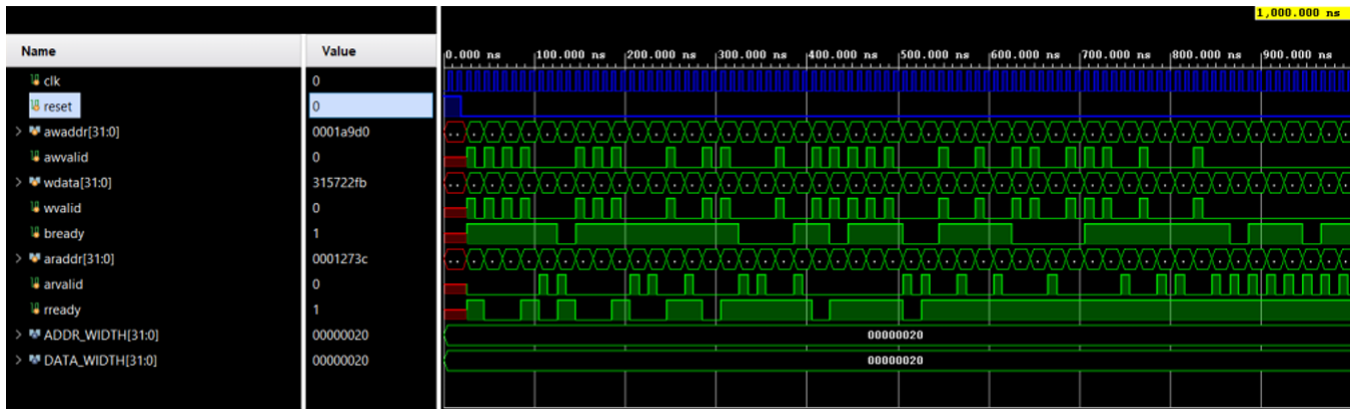
These are the screenshot captured during the end of the project.

Figure 9.1 Vivado elaborated RTL schematic



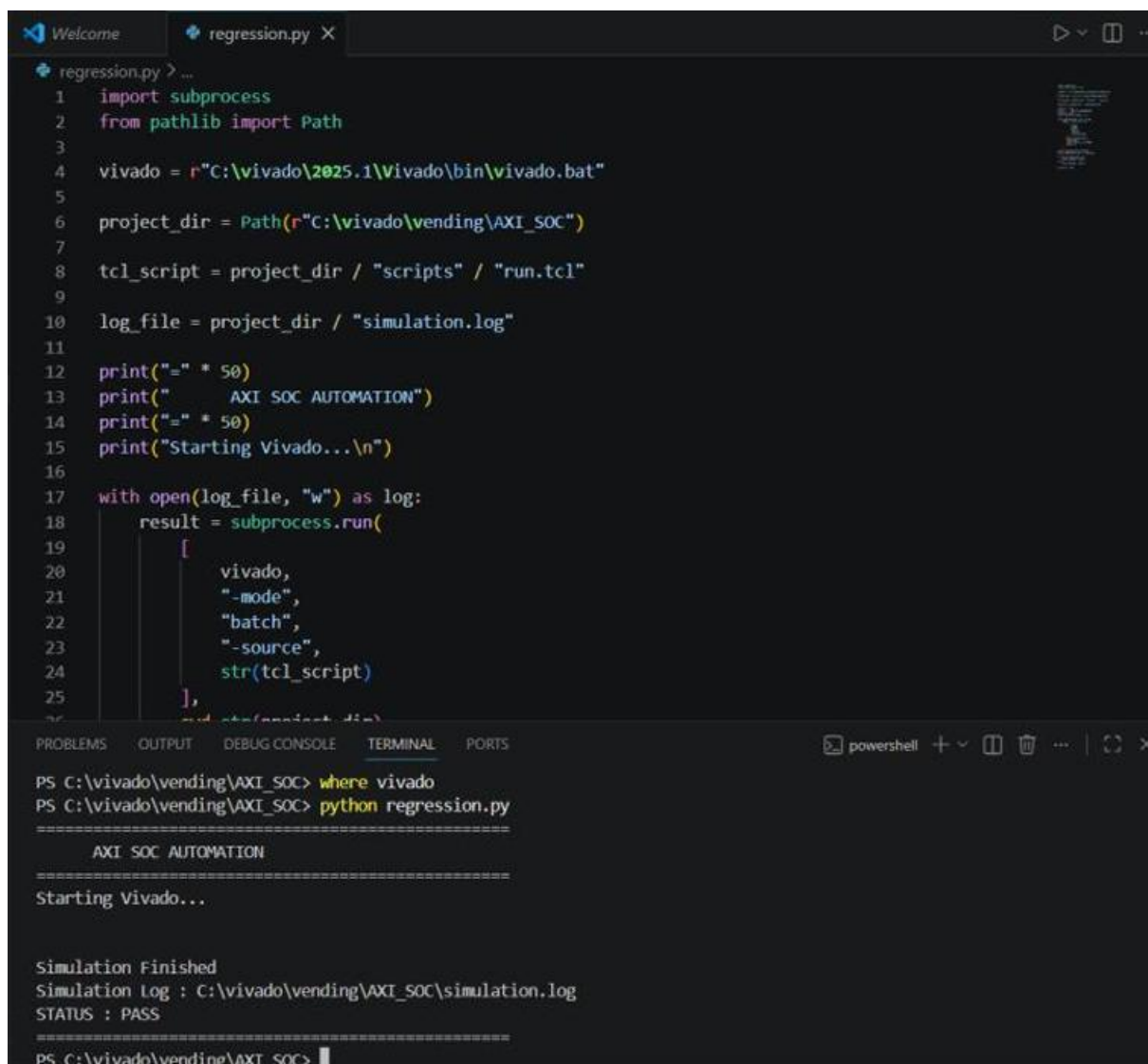
Elaborated RTL schematic of the design with 178 cells, 2 I/O ports and 835 nets.

Figure 9.2 Vivado Behavioural simulation waveform



Waveform capture of signals over 1000ns.

Figure 9.3 Python / Vivado batch regression concole



```

1  import subprocess
2  from pathlib import Path
3
4  vivado = r"C:\vivado\2025.1\ Vivado\bin\vivado.bat"
5
6  project_dir = Path(r"C:\vivado\ vending\AXI_SOC")
7
8  tcl_script = project_dir / "scripts" / "run.tcl"
9
10 log_file = project_dir / "simulation.log"
11
12 print("=" * 50)
13 print("      AXI SOC AUTOMATION")
14 print("=" * 50)
15 print("Starting Vivado...\n")
16
17 with open(log_file, "w") as log:
18     result = subprocess.run(
19         [
20             vivado,
21             "-mode",
22             "batch",
23             "-source",
24             str(tcl_script)
25         ],
26         cwd=project_dir,
27     )

```

```

PS C:\vivado\ vending\AXI_SOC> where vivado
PS C:\vivado\ vending\AXI_SOC> python regression.py
=====
      AXI SOC AUTOMATION
=====
Starting Vivado...

Simulation Finished
Simulation Log : C:\vivado\ vending\AXI_SOC\simulation.log
STATUS : PASS
=====
PS C:\vivado\ vending\AXI_SOC>

```

VS Code terminal output from regression.py driving Vivado in -mode batch showing Simulation Finished and STATUS : PASS.

Figure 9.4 UVM Testbench Scoreboard / Coverage Console Output

```

FUNCTIONAL COVERAGE = 100.00 %
cp_write    = 100.00
cp_read     = 100.00
cp_awaddr   = 100.00
cp_araddr   = 100.00
cp_wdata    = 100.00
cp_bready   = 100.00
cp_rready   = 100.00

```

```
--- UVM Report Summary ---
```

```
** Report counts by severity
```

```

UVM_INFO :    2
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0

```

EDA playground (VCS) console output showing 100% functional coverage.

Figure 9.5 Class Based Testbench Scoreboard / Coverage Console Output

```

=====
FUNCTIONAL COVERAGE = 100.00 %
cp_write    = 100.00
cp_read     = 100.00
cp_awaddr   = 100.00
cp_araddr   = 100.00
cp_wdata    = 100.00
cp_bready   = 100.00
cp_rready   = 100.00
=====

```

EDA playground (VCS) console output showing same 100% functional coverage.

9.4 Key Observations

- Both independently-coded verification environments converge on identical results: 100% functional coverage, zero errors, and all directed corner cases (address 0, mid, and top-of-space) passing.
- No SVA property failed during any regression run, indicating the handshake, arbitration, and FIFO pointer logic are protocol-compliant under both random and directed stimulus.
- The class-based regression's much larger transaction volume (30,000 vs. 200) provided additional confidence in the design's behavior under sustained, high-throughput contention.

10. Build & Run

10.1 Vivado Batch Simulation (Tcl)

The following Tcl script drives Vivado's simulator in batch mode (scripts/run.tcl):

```
open_project AXI_SOC.xpr
update_compile_order -fileset sim_1
launch_simulation
restart
log_wave -r /*
run all
puts "Simulation Completed Successfully"
exit
```

10.2 Python Regression Automation

scripts/regression.py wraps the Tcl script in a headless Vivado invocation and reports pass/fail based on the process return code:

```
import subprocess
from pathlib import Path

vivado = r"C:\vivado\2025.1\Vivado\bin\vivado.bat"
project_dir = Path(r"C:\vivado\wending\AXI_SOC")
tcl_script = project_dir / "scripts" / "run.tcl"
log_file = project_dir / "simulation.log"

print("=" * 50)
print("      AXI SOC AUTOMATION")
print("=" * 50)
print("Starting Vivado...\n")

with open(log_file, "w") as log:
    result = subprocess.run(
        [vivado, "-mode", "batch", "-source", str(tcl_script)],
        cwd=str(project_dir),
        stdout=log,
        stderr=subprocess.STDOUT,
        text=True,
    )

print("\nSimulation Finished")
print("Simulation Log :", log_file)
print("STATUS :", "PASS" if result.returncode == 0 else "FAIL")
print("=" * 50)
```

Run it from a terminal / VS Code integrated terminal with:

```
python regression.py
```

10.3 UVM / Class-Based Simulation (EDA Playground or any VCS-class simulator)

```
# UVM environment
vlog +incdir+. tb/uvm/*.sv rtl/*.v
vsim -c work.tb_top -do "run -all; quit"

# Class-based environment
```

```
vlog +incdir+. tb/class_based/*.sv rtl/*.v
vsim -c work.tb_top -do "run -all; quit"
```

The regression for this project was executed on EDA Playground using a VCS-class simulator for both the UVM and class-based testbenches, and on Vivado 2025.1 for elaboration, schematic review, and waveform-based behavioral simulation.

11. Tools Used

Category	Tool
RTL & Elaboration / Synthesis	Xilinx Vivado 2025.1
UVM / Class-Based Simulation	EDA Playground (VCS cloud simulator)
Waveform Viewing	Vivado Simulator waveform viewer / \$dumpfile-\$dumpvars VCD
Automation Scripting	Python 3.14 (subprocess-based Vivado batch driver)
Simulator Control Scripting	Tcl (Vivado batch-mode simulation script)
Editors	Visual Studio Code (Python automation), Notepad (Tcl script authoring)
Version Control	Git (recommended for repository hosting)
Languages	Verilog (RTL), SystemVerilog (assertions, UVM and class-based testbenches)

12. Future Work

- Extend the interconnect to a fuller AXI4 subset: burst transactions (AWLEN/ARLEN), byte-strobes (WSTRB), and response encoding (BRESP/RRESP).
- Add per-master address decoding / memory maps instead of a single shared 256-word slave memory.
- Replace the fixed round-robin arbiter with a weighted or QoS-aware scheme and add starvation/latency coverage.
- Introduce true independent wr_clk/rd_clk domains for the asynchronous FIFO and add CDC-focused formal or simulation-based verification.
- Add a UVM register abstraction layer (RAL) model for the slave's memory-mapped space.
- Port the design to an FPGA target for hardware-in-the-loop validation alongside the existing simulation-based verification.
- Integrate code-coverage (line/toggle/FSM) alongside the existing functional coverage to strengthen verification closure metrics.

13. Author

Gautam Raj Chhabra

- B.Tech, Electronics and Communication Engineering (ECE) — expected graduation 2028.
- Focus areas: RTL Design (Verilog), Functional Verification (SystemVerilog / UVM), AXI-style Interconnect and Arbitration Design, Asynchronous FIFO / CDC fundamentals.

- Tooling experience: Xilinx Vivado, EDA Playground (VCS), Python automation for EDA regression, Tcl scripting.
- This project was built end-to-end: RTL authoring, dual testbench development (UVM and class-based), assertion-based checking, regression automation, and results documentation.

LinkedIn : <https://www.linkedin.com/in/grajchhabra>

GitHub : <https://github.com/grajchhabra-sketch>

Gmail : grajchhabra@gmail.com